МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №6

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Файли, що відображаються в пам'ять в ОС Windows

Мета. Ознайомитися з відображенням файлів в оперативну пам’ять в ОС Windows. Навчитися реалізовувати відображення файлів в оперативну пам’ять.

Завдання.

1. Реалізувати відображення файлу в оперативну пам'ять.
2. Необхідно модифікувати **лабораторну роботу №4** таким чином, щоб всі результати роботи програми записувалися через відображення файлів в оперативній пам’яті.
3. Виконати завдання, з використанням цього відображення.
4. Результати виконання роботи відобразити у звіті.

Варіант 6

6. Створити масив N елементів і відсортувати його елементи у порядку ***спадання*** за допомогою методу ***«вибірки»***. Елементи масиву згенерувати випадковим чином за допомогою вбудованих функцій. Результуючий масив вивести у зовнішній файл через відображення цього файлу в оперативній пам’яті.

Код реалізації програми:

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <windows.h>
#include <algorithm>
#include <vector>
#include <ctime>

#define N 10

using namespace std;

HANDLE hThreads[8];
HANDLE hThreadOpen[8];
HANDLE hMutex;

vector <int> ThreadFuncTakeNums;

volatile LONG TotalThreadCount = 0;

DWORD WINAPI selectionSort(void* param) {
    vector<int>& temp = *(vector<int>*) param;
    int min;

    WaitForSingleObject(hMutex, INFINITE);
    for (int i = 0; i < temp.size(); i++) {
        ThreadFuncTakeNums.push_back(temp[i]);
    }

    for (int i = 0; i < temp.size() - 1; i++) {

        min = i;
```

```
            min = i;
            for (int j = i + 1; j < temp.size(); j++)
                if (temp[j] < temp[min])
                    min = j;
            swap(temp[i], temp[min]);

        }
        ReleaseMutex(hMutex);

        return 0;
    }

vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; i++) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
    return completed;
}

void showNumbers(vector<int> vec) {
    for (auto now : vec) {
```

```
    for (auto now : vec) {
        cout << now << " ";
    }
    cout << endl;
}

int main() {
    srand(time(NULL));

    char* pItoa;
    char foo[2];

    DWORD dwIncre;

    //creating mutex
    hMutex = CreateMutex(
        NULL,
        FALSE,
        NULL
    );

    int thread_num;
    vector<vector<int>> arr;

    vector<int> input(N);
    for (int i = 0; i < N; ++i) {
        input[i] = rand() % N + 1;
    }

    //count the size of file, which we will create
    int fileSize = 0;
```

```
    int fileSize = 0;
    for (int i = 0; i < N; i++)
    {
        input[i] = rand() % (N * 10);
        for (pItoa = _itoa(input[i], foo, 10); *pItoa; pItoa++, fileSize++);
    }
    fileSize += N;

    cout << "The massive before sorting:" << endl;
    showNumbers(input);

    cout << "Enter the number of threads: ";
    cin >> thread_num;
    for (int i = 0; i < thread_num; ++i) {
        vector<int> tmp;
        for (int j = i * N / thread_num; j < (i + 1) * N / thread_num; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    int last_interation = 0;

    for (int i = 0; i < thread_num; i++) {
        int ThreadPriority;

        DWORD ThreadID;

        HANDLE thread = CreateThread(NULL, 0, selectionSort, &arr[i], 0, &ThreadID);
        hThreads[i] = thread;

```

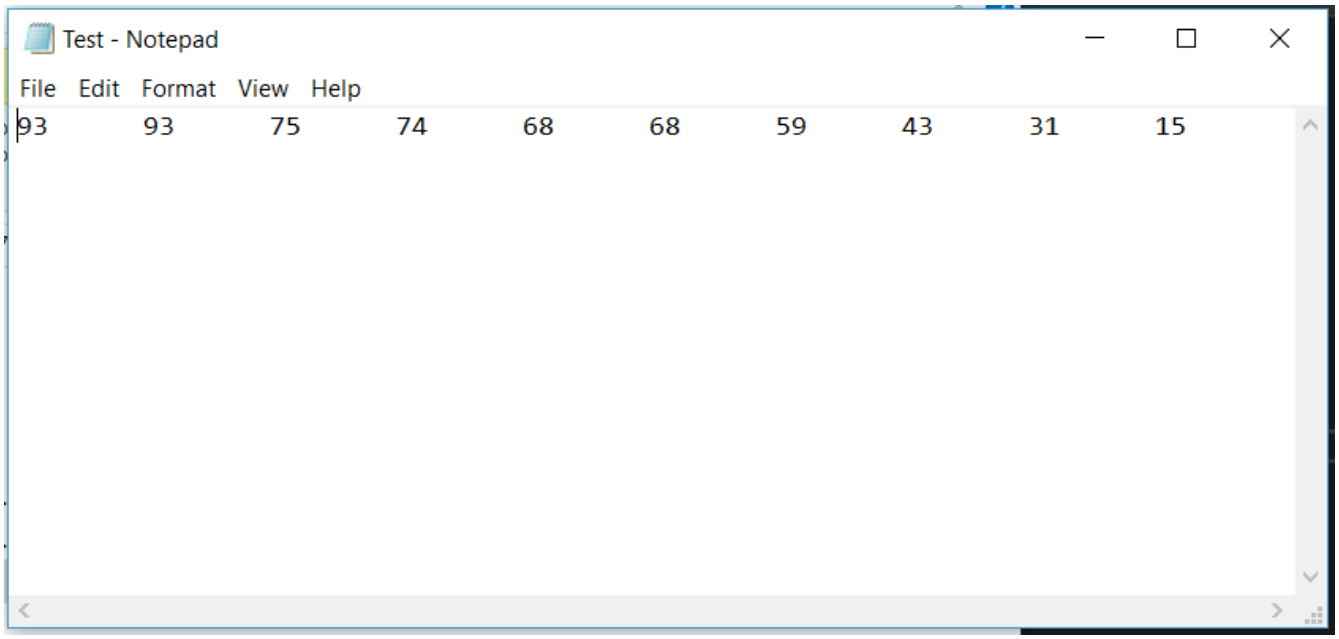
```
        cout << "Number of thread - " << i + 1 << endl;

        //ThreadID
        cout << "Thread ID = " << ThreadID << endl;

        cout << "Values, which take this " << ThreadID << " thread:" << endl;

        for (int k = last_interation; k < ThreadFuncTakeNums.size(); k++)
        {
            cout << ThreadFuncTakeNums[k] << " ";
        }
        cout << endl;

        if (hThreads[i] == NULL)
        {
            cout << "Thread Error" << endl;
            ExitProcess(1);
        }

        //Opend Thread
        HANDLE hOpenThread = OpenThread(
            THREAD_SET_LIMITED_INFORMATION,
            FALSE,
            ThreadID
        );

        //interlock function
        dwIncre = InterlockedIncrement((LPLONG)&TotalThreadCount);

        hThreadOpen[i] = hOpenThread;
        if (NULL == hOpenThread)
        {
            cout << "Open Thread Function Failed & Error - " << GetLastError() << endl;
            ExitProcess(2);
        }

        // Set Thread Priority
        int mode;
        cout << "Set priority: " << endl;
        cout << "Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = ";
        cin >> mode;

        while (mode > 7 || mode < 0)
        {
            cin >> mode;
        }

        switch (mode) {
        case 1:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_LOWEST);
            break;
        case 2:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_BELOW_NORMAL);
            break;
        case 3:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_NORMAL);
            break;
        case 4:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_ABOVE_NORMAL);
            break;
        case 5:
        case 5:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_HIGHEST);
            break;
        case 6:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_TIME_CRITICAL);
            break;
        }

        //Getting Thread Priority
        ThreadPriority = GetThreadPriority(thread);

        cout << "Thread Priority = " << ThreadPriority << endl;

        //Closing Handle of Thread
        CloseHandle(thread);

        cout << "Interlock increment = " << dwIncre << endl;

        last_interation += ThreadFuncTakeNums.size();
        cout << endl;
    }

    for (int i = 0; i < thread_num; ++i) {
        //WaitForSingleObject for Thread
        WaitForSingleObject(hThreadOpen[i], INFINITE);
        CloseHandle(hThreadOpen[i]);
    }

    vector<int> completed = complete(arr, N, thread_num);
    cout << "Sorting number:" << endl;
    showNumbers(completed);
```

```
    showNumbers(completed);

    HANDLE hFile, hMap;
    LPBYTE pData = NULL; //pointer on start of the file

    hFile = CreateFile(
        L"C:\\Users\\User\\source\\repos\\717\\717\\Test.txt",
        GENERIC_WRITE | GENERIC_READ,
        0,
        NULL,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );

    //check if hangle of file is created
    if (hFile != INVALID_HANDLE_VALUE) {
        //create projection of all file
        hMap = CreateFileMapping(hFile, NULL, PAGE_READWRITE, 0, fileSize, NULL);
        if(hMap) {
            DWORD dwSize = GetFileSize(hFile, NULL);
            CloseHandle(hFile);

            cout << "Size of file = " << dwSize << "bytes" << endl;

            //project on address space
            pData = (LPBYTE)MapViewOfFile(hMap, FILE_MAP_WRITE, 0, 0, 0);

            if (pData) {
                int offset = 0;
                for (int i = 0; i < N; i++, offset++) {
                    for (pItoa = _itoa(completed[i], foo, 10); *pItoa; pItoa++, offset++)
                        *(pData + offset) = *pItoa;
                    *(pData + offset) = '\t';
                }
                //close project on address space
                UnmapViewOfFile(pData);
            }
            CloseHandle(hMap);
        }

        cout << endl;
        cout << "Resulting massive is in file - Test.txt" << endl;
        cout << endl;
    }

    system("pause");
    return 0;
}
```

Реалізація програми у console:

```
C:\Users\User\source\repos\717\Debug\717.exe

The massive before sorting:
43 31 15 93 75 68 93 68 74 59
Enter the number of threads: 2
Number of thread - 1
Thread ID = 10264
Values, which take this 10264 thread:

Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 1

Number of thread - 2
Thread ID = 1224
Values, which take this 1224 thread:
68 93 68 74 59
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 2

Sorting number:
93 93 75 74 68 68 59 43 31 15
Size of file = 30bytes

Resulting massive is in file - Test.txt

Press any key to continue . . .
```

Запис даних у файл:



Висновок: я ознайомився з відображенням файлів в оперативну пам’ять в ОС Windows. Навчитися реалізовувати відображення файлів в оперативну пам’ять.